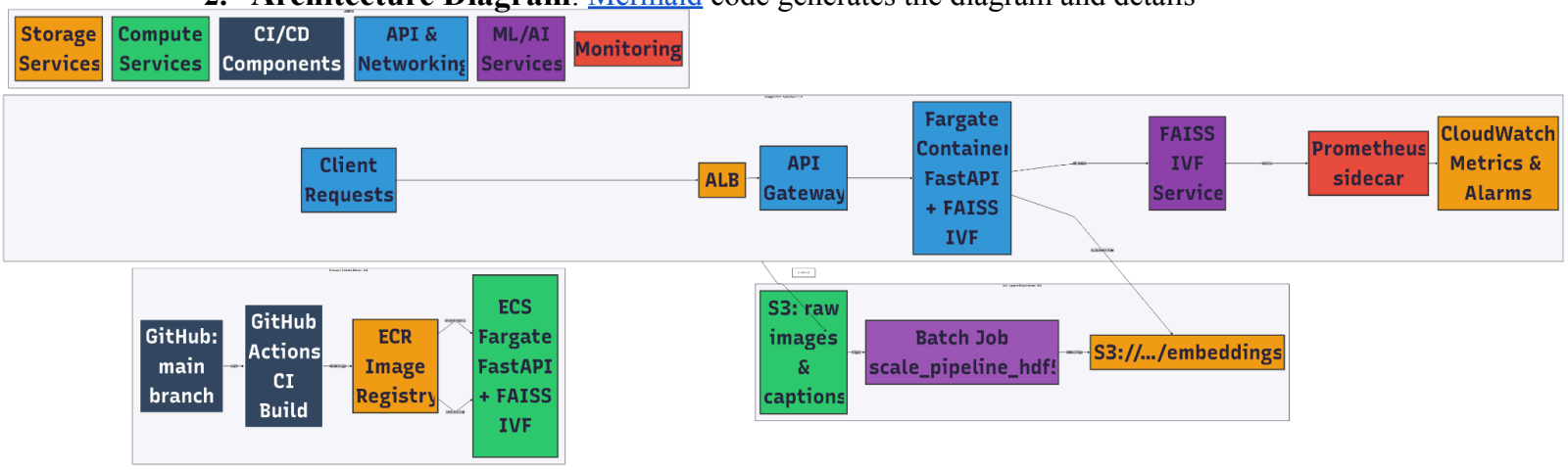
1. **Introduction**

   In step 9, we settled on an AWS-native pay-per-use deployment stack using ECS fargate for inference, S3 for storage, and GitHub CI/CD for automated builds. In step 10, we turn that prototype into a production system. We present an end-to-end architecture diagram; describe each major component and how data is transferred between them; outline the model's life cycle; specify monitoring, logging, and rollback procedures; list the concrete tools and services we will use; and find an estimate for implementation costs.

2. **Architecture Diagram**: Mermaid code generates the diagram and details



3. **Major Components and Data Flow**

   1. Data Ingestion & ETL

   Source: Raw images & captions land in an S3 "ingest" bucket. **Batch Job**: A daily AWS Batch (or Step Functions) job runs our chunked embedding pipeline (`scale_pipeline_hdf5.py`) against the full dataset. **Output**: Resulting image/text embeddings are written to S3 (`s3://.../embeddings/embeddings_full.h5`).

   2. CI/CD & Container Registry

   Source Control: Pushes to `main` in GitHub trigger a GitHub Actions workflow. **Build**: The workflow builds a Docker image that bundles our FastAPI inference service & FAISS IVF index. **Registry**: The image is pushed to Amazon ECR.

   3.Inference Service

   ECS Fargate: On every image push, ECS automatically deploys the new container revision. Fargate tasks run behind an Application Load Balancer (ALB). **API Gateway**: Sits in front of the ALB to handle authorization, throttling, and request routing. **FAISS IVF**: The container loads the IVF index from S3 or local EFS on startup, then serves nearest-neighbor lookups.

4. Monitoring & Metrics

   Prometheus Side-car: Collects custom metrics (latency, QPS, error rates) from the FastAPI/FAISS service. **CloudWatch**: Aggregates logs, system metrics, and the Prometheus endpoint. Alarms fire on high latency (>150 ms p95), low recall canary failures, or container crashes.

4. Model Lifecycle

1. Initial Training: We precomputed embeddings and built the IVF index offline.
2. Retraining Cadence:
    a. **Scheduled**: A weekly pipeline runs on new data to refresh embeddings and rebuild the index.
    b. **Conditional**: We'll monitor a "recall drift" canary; if recall on a validation set falls below 80%, it triggers an immediate rebuild.
3. Artifact Management:
    a. **Embeddings & Index**: Stored in S3 under versioned prefixes (e.g. `s3://.../embeddings/v2025-07-01/embeddings_full.h5`).
    b. **Container Image**: Tagged in ECR with the same version.
4. Deployment: After passing evaluation, a GitHub Actions job promotes the container tag to "prod" and updates the ECS service (rolling deploy).
5. Rollback: We maintain Terraform-managed `deploy_version` that can be pointed at the previous, validated ECR tag.

5. Tools & Technologies

Layer	Service / Tool	Purpose
Compute	AWS Fargate	Serverless containers for inference
Storage	Amazon S3	Embedding & model artifacts; static assets
CI/CD	GitHub Actions & ECR	Build & push Docker images
Batch ETL	AWS Batch / Step function	Offline embedding & index rebuild
API Front-end	ALB/API Gateway	Load balancing, authorization, throttling
Monitoring	Prometheus & CW Alarms	Latency, recall canaries, resource usage
Orchestration	Terraform	Infrastructure as code

6. Implementation Effort & Estimated Cost

The effort for this should approximately take 4-8 hours to wire up the Terraform modules, CI/CD pipelines, and monitoring dashboards, The run rate includes: ECS Fargate pay-per-second CPU/memory approx. $2/day under moderate load, S3 storage approx. $0.50.month for embeddings and static assets, batch GPU jobs.

7. Scalability, Fault-Tolerance & Edge Cases

Autoscaling: ECS Service auto-scales Fargate tasks based on CPU, memory, & latency. **Mult-AZ**: ALB routes across tasks in multiple AZs for high availability. **Graceful Failure**: Container health checks & retry logic in API Gateway. **Data Backfill**: Historical reprocessing via Batch jobs for late-arriving or corrected data. **Versioning**: S3 prefixes and ECR tages ensure reproducible rollbacks.